

1. Estructura de Carpetas Actualizada

Se añaden los módulos de mapa y entidad bastión. Los archivos existentes se mantienen sin cambios (excepto `states.py` y `config.py`, que se modifican ligeramente).

```
...
el-ultimo-bastion/
├── main.py           (sin cambios)
├── config.py         ← MODIFICADO (nuevas constantes de mapa)
├── src/
│   ├── core/
│   │   ├── __init__.py
│   │   ├── game.py   (sin cambios)
│   │   └── states.py  ← MODIFICADO (GameState ahora renderiza el escenario)
│   ├── entities/
│   │   ├── __init__.py ← NUEVO (vacío)
│   │   └── bastion.py  ← NUEVO
│   ├── map/
│   │   ├── __init__.py ← NUEVO (vacío)
│   │   ├── tile.py     ← NUEVO
│   │   ├── path.py     ← NUEVO
│   │   └── grid.py     ← NUEVO
│   ├── managers/      (vacío, no se modifica)
│   ├── ui/            (vacío, no se modifica)
│   └── utils/         (vacío, no se modifica)
├── assets/           (vacío, se usarán colores planos por ahora)
├── docs/
├── tests/
└── data/
...
```

2. Archivos Creados y Modificados – Código Completo

2.1 `config.py` (modificado)

Se añaden constantes para la cuadrícula y colores del terreno.

```
```python
"""
Configuración global del videojuego "El Último Bastión".
"""

Ventana
SCREEN_WIDTH = 1024
SCREEN_HEIGHT = 768
FPS = 60
GAME_TITLE = "El Último Bastión"
```
```

```

# Colores básicos (RGB)
COLOR_BLACK = (0, 0, 0)
COLOR_WHITE = (255, 255, 255)
COLOR_RED = (255, 0, 0)
COLOR_GREEN = (0, 255, 0)
COLOR_BLUE = (0, 0, 255)
COLOR_GREY = (100, 100, 100)
COLOR_DARK_BG = (20, 20, 40)
COLOR_MENU_TEXT = (220, 220, 220)

# Configuración de la cuadrícula del mapa
TILE_SIZE = 64          # Tamaño de cada celda en píxeles
GRID_COLS = 16          # Número de columnas (1024 / 64)
GRID_ROWS = 12          # Número de filas (768 / 64)

# Tipos de tile
TILE_GRASS = 0          # Construible
TILE_PATH = 1           # Camino (no construible)
TILE_ROCK = 2           # Bloqueado, no construible
TILE_BASTION = 3        # Posición del bastión

# Colores del terreno (temporales, hasta tener sprites)
COLOR_GRASS = (34, 139, 34) # Verde bosque
COLOR_PATH = (139, 90, 43)  # Marrón tierra
COLOR_ROCK = (105, 105, 105) # Gris oscuro
COLOR_BASTION_TILE = (80, 80, 80) # Gris para el tile del bastión
COLOR_BASTION = (180, 180, 180) # Color de la estructura del bastión
COLOR_BUILDABLE_HIGHLIGHT = (0, 200, 0, 100) # Verde semitransparente (para futuro)
'''

```

2.2 `src/map/tile.py` (nuevo)

```

'''python
'''

Representación de una celda del mapa.
'''

from config import TILE_GRASS, TILE_PATH, TILE_ROCK, TILE_BASTION

class Tile:
    '''
    Una celda individual del mapa.
    '''

    def __init__(self, tile_type=TILE_GRASS):
        self.tile_type = tile_type    # Tipo de terreno
        self.defense = None          # Referencia a una defensa construida (None = vacío)

```

```

def is_buildable(self):
    """Devuelve True si se puede construir una defensa sobre esta celda."""
    return self.tile_type == TILE_GRASS and self.defense is None

def is_walkable(self):
    """Devuelve True si un enemigo puede caminar por esta celda."""
    # El camino es transitable por enemigos, el resto no.
    return self.tile_type == TILE_PATH

def is_bastion(self):
    return self.tile_type == TILE_BASTION
...

### 2.3 `src/map/path.py` (nuevo)

```python
"""
Define el camino de waypoints que seguirán los enemigos.
"""

import pygame
from config import TILE_SIZE

class Path:
 """
 Almacena una lista de puntos (Vector2) que forman el recorrido.
 """
 def __init__(self, waypoints):
 """
 Args:
 waypoints: lista de tuplas (x, y) en píxeles (centro de la celda).
 """
 self.waypoints = [pygame.Vector2(x, y) for x, y in waypoints]

 def get_start(self):
 """Devuelve el primer waypoint (spawn de enemigos)."""
 if self.waypoints:
 return self.waypoints[0]
 return None

 def get_end(self):
 """Devuelve el último waypoint (cerca del bastión)."""
 if self.waypoints:
 return self.waypoints[-1]
 return None

 def get_waypoint(self, index):
 """Devuelve el waypoint en la posición dada, o None si está fuera de rango."""

```

```

 if 0 <= index < len(self.waypoints):
 return self.waypoints[index]
 return None

 def total_waypoints(self):
 return len(self.waypoints)
...

2.4 `src/map/grid.py` (nuevo)

```python
"""
Matriz de tiles que compone el mapa del juego.
"""

import pygame
from config import TILE_SIZE, GRID_COLS, GRID_ROWS, COLOR_GRASS,
COLOR_PATH, COLOR_ROCK, COLOR_BASTION_TILE
from src.map.tile import Tile, TILE_GRASS, TILE_PATH, TILE_ROCK, TILE_BASTION

class Grid:
    """
    Cuadrícula del mapa.
    """
    def __init__(self, layout):
        """
        Args:
            layout: matriz 2D (filas x columnas) de enteros (tipo de tile).
        """
        self.rows = GRID_ROWS
        self.cols = GRID_COLS
        self.tiles = []
        for row in range(self.rows):
            tile_row = []
            for col in range(self.cols):
                tile_type = layout[row][col]
                tile_row.append(Tile(tile_type))
            self.tiles.append(tile_row)

    def get_tile(self, row, col):
        """Devuelve el Tile en (row, col) o None si está fuera de límites."""
        if 0 <= row < self.rows and 0 <= col < self.cols:
            return self.tiles[row][col]
        return None

    def get_pixel_rect(self, row, col):
        """Devuelve un pygame.Rect para la celda (row, col)."""
        x = col * TILE_SIZE

```

```

        y = row * TILE_SIZE
        return pygame.Rect(x, y, TILE_SIZE, TILE_SIZE)

    def draw(self, screen):
        """
        Dibuja cada celda con el color correspondiente.
        """
        for row in range(self.rows):
            for col in range(self.cols):
                tile = self.tiles[row][col]
                rect = self.get_pixel_rect(row, col)
                if tile.tile_type == TILE_GRASS:
                    color = COLOR_GRASS
                elif tile.tile_type == TILE_PATH:
                    color = COLOR_PATH
                elif tile.tile_type == TILE_ROCK:
                    color = COLOR_ROCK
                elif tile.tile_type == TILE_BASTION:
                    color = COLOR_BASTION_TILE
                else:
                    color = (0, 0, 0) # fallback
                pygame.draw.rect(screen, color, rect)
                # Borde sutil para cada celda
                pygame.draw.rect(screen, (50, 50, 50), rect, 1)

    def get_bastion_position(self):
        """
        Busca y devuelve la celda (row, col) donde está el bastión.
        """
        for row in range(self.rows):
            for col in range(self.cols):
                if self.tiles[row][col].tile_type == TILE_BASTION:
                    return row, col
        return None
...

```

2.5 `src/entities/bastion.py` (nuevo)

```

'''python
"""
Clase que representa el Bastión (corazón de la fortaleza).
"""

import pygame
from config import TILE_SIZE, COLOR_BASTION

class Bastion:
    """

```

El Bastión tiene una posición fija y una barra de vida.

"""

```
def __init__(self, grid_row, grid_col, max_health=100):
```

"""

Args:

grid_row, grid_col: posición en la cuadrícula.

max_health: vida máxima inicial.

"""

```
self.grid_row = grid_row
```

```
self.grid_col = grid_col
```

```
self.max_health = max_health
```

```
self.health = max_health
```

```
# Posición en píxeles (centro de la celda)
```

```
self.x = grid_col * TILE_SIZE + TILE_SIZE // 2
```

```
self.y = grid_row * TILE_SIZE + TILE_SIZE // 2
```

```
self.radius = TILE_SIZE // 2 - 4 # tamaño visual
```

```
def take_damage(self, amount):
```

```
    """Reduce la vida del bastión."""
```

```
    self.health = max(0, self.health - amount)
```

```
def is_destroyed(self):
```

```
    return self.health <= 0
```

```
def draw(self, screen):
```

```
    """Dibuja el bastión como un círculo y una barra de vida sencilla."""
```

```
    # Círculo central
```

```
    pygame.draw.circle(screen, COLOR_BASTION, (self.x, self.y), self.radius)
```

```
    # Borde
```

```
    pygame.draw.circle(screen, (100, 100, 100), (self.x, self.y), self.radius, 2)
```

```
    # Barra de vida
```

```
    bar_width = self.radius * 2
```

```
    bar_height = 6
```

```
    bar_x = self.x - bar_width // 2
```

```
    bar_y = self.y - self.radius - 10
```

```
    # Fondo rojo
```

```
    pygame.draw.rect(screen, (200, 0, 0), (bar_x, bar_y, bar_width, bar_height))
```

```
    # Relleno verde proporcional a la vida
```

```
    fill_width = int(bar_width * (self.health / self.max_health))
```

```
    if fill_width > 0:
```

```
        pygame.draw.rect(screen, (0, 200, 0), (bar_x, bar_y, fill_width, bar_height))
```

```
    # Borde de la barra
```

```
    pygame.draw.rect(screen, (255, 255, 255), (bar_x, bar_y, bar_width, bar_height), 1)
```

```
...
```

2.6 `src/core/states.py` (modificado – solo se muestra la parte de GameplayState; el resto permanece igual)

Se incluye el código completo del archivo para contexto, aunque las clases `MenuState`, `PauseState` y `GameOverState` no cambian. Se resalta la nueva versión de `GameplayState`.

```
```python
"""
Máquina de estados del juego "El Último Bastión".
"""

import pygame
from config import (
 COLOR_DARK_BG, COLOR_MENU_TEXT, COLOR_WHITE, COLOR_BLUE,
 COLOR_RED, COLOR_BLACK,
 GRID_COLS, GRID_ROWS, TILE_SIZE,
 TILE_GRASS, TILE_PATH, TILE_ROCK, TILE_BASTION
)
from src.map.grid import Grid
from src.map.path import Path
from src.entities.bastion import Bastion

----- Clase base State (sin cambios) -----
class State:
 def __init__(self, game):
 self.game = game

 def handle_events(self, event):
 raise NotImplementedError

 def update(self, dt):
 raise NotImplementedError

 def render(self, screen):
 raise NotImplementedError

----- MenuState (sin cambios) -----
class MenuState(State):
 def __init__(self, game):
 super().__init__(game)
 self.title_font = pygame.font.Font(None, 72)
 self.option_font = pygame.font.Font(None, 48)
 self.title_text = self.title_font.render("EL ÚLTIMO BASTIÓN", True,
COLOR_MENU_TEXT)
 self.play_text = self.option_font.render("Presiona ENTER para Jugar", True,
COLOR_WHITE)
 self.quit_text = self.option_font.render("Presiona ESC para Salir", True,
COLOR_WHITE)
 screen_center_x = self.game.screen.get_width() // 2
```

```

 screen_center_y = self.game.screen.get_height() // 2
 self.title_rect = self.title_text.get_rect(center=(screen_center_x, screen_center_y - 100))
 self.play_rect = self.play_text.get_rect(center=(screen_center_x, screen_center_y +
20))
 self.quit_rect = self.quit_text.get_rect(center=(screen_center_x, screen_center_y + 80))

 def handle_events(self, event):
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_RETURN:
 self.game.change_state(GameplayState(self.game))
 elif event.key == pygame.K_ESCAPE:
 self.game.running = False

 def update(self, dt):
 pass

 def render(self, screen):
 screen.fill(COLOR_DARK_BG)
 screen.blit(self.title_text, self.title_rect)
 screen.blit(self.play_text, self.play_rect)
 screen.blit(self.quit_text, self.quit_rect)

----- GameplayState (MODIFICADO) -----
class GameplayState(State):
 """
 Estado principal de juego.
 Contiene el mapa, el camino y el bastión.
 """
 def __init__(self, game):
 super().__init__(game)
 # --- Definir el layout del mapa ---
 # Matriz de 12 filas x 16 columnas.
 # 0 = pasto (construible), 1 = camino, 2 = roca (bloqueado), 3 = bastión
 layout = [
 [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
]

```



```

Definir un camino sinuoso sobre la cuadrícula (modificamos el layout)
Colocamos tiles de camino desde la izquierda hacia el bastión en la derecha.
Camino horizontal en fila 1, luego vertical en columna 7, luego horizontal fila 10, etc.
Vamos a reescribir el layout con un camino más interesante.
Para mayor claridad, definimos el layout final ya con el camino incluido.
self.layout = [
 [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2],
 [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
]
Agregar bastión: lo colocamos en la fila 7, columna 13 (cerca del final del camino)
self.layout[7][13] = TILE_BASTION
Ajustar el camino para que llegue hasta el bastión: ponemos tiles de camino desde
columna 6 a 13 en la fila 7
for col in range(6, 13):
 self.layout[7][col] = TILE_PATH
self.layout[7][13] = TILE_BASTION # el último tile es el bastión

Crear Grid y Path
self.grid = Grid(self.layout)

Waypoints: lista de centros de celdas del camino (para que los enemigos sigan)
Extraemos automáticamente del layout las celdas que son TILE_PATH + el
TILE_BASTION al final.
waypoints_px = []
for row in range(GRID_ROWS):
 for col in range(GRID_COLS):
 if self.layout[row][col] == TILE_PATH or self.layout[row][col] == TILE_BASTION:
 x = col * TILE_SIZE + TILE_SIZE // 2
 y = row * TILE_SIZE + TILE_SIZE // 2
 waypoints_px.append((x, y))
Ordenar waypoints según el recorrido lógico (izquierda a derecha, luego abajo, etc.)
En este caso, el camino es:
Entrada: fila 7, columna 1 (izquierda) -> avanza hacia la derecha por fila 7 hasta
columna 13.
Así que los waypoints en orden natural de columna creciente en la fila 7 son
correctos.
Además, añadimos un tramo vertical opcional: más adelante se puede complicar.
self.path = Path(waypoints_px)

```

```

Crear bastión en la celda correspondiente
bastion_pos = self.grid.get_bastion_position()
if bastion_pos:
 self.bastion = Bastion(*bastion_pos, max_health=100)
else:
 # Fallback
 self.bastion = Bastion(7, 13, 100)

Fuente para información adicional
self.font = pygame.font.Font(None, 28)
self.info_text = self.font.render("Escenario base - ESC para pausa", True,
COLOR_WHITE)

def handle_events(self, event):
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_ESCAPE:
 from src.core.states import PauseState
 self.game.change_state(PauseState(self.game, self))

def update(self, dt):
 # En futuras iteraciones: movimiento de enemigos, ataques, etc.
 pass

def render(self, screen):
 # Fondo base (si no se cubre con el grid)
 screen.fill((0, 0, 0))
 # Dibujar la cuadrícula
 self.grid.draw(screen)
 # Dibujar el bastión
 self.bastion.draw(screen)
 # Texto informativo (temporal)
 screen.blit(self.info_text, (10, 10))
...

```

\*Nota: Las clases `PauseState` y `GameOverState` permanecen idénticas a la entrega anterior. Se omite su repetición por brevedad.\*

### 2.7 Archivos `\_\_init\_\_.py` (nuevos, vacíos)

- `src/entities/\_\_init\_\_.py`: archivo vacío.
- `src/map/\_\_init\_\_.py`: archivo vacío.

---

## 3. Explicación Técnica Detallada

### Mapa y cuadrícula (`Grid`, `Tile`)

- **Tile**: representa una celda individual con un tipo (`TILE_GRASS`, `TILE_PATH`, `TILE_ROCK`, `TILE_BASTION`) y un campo `defense` (de momento `None`). Los métodos `is_buildable()`, `is_walkable()` preparan la lógica futura.
- **Grid**: recibe una matriz de enteros (definida manualmente en `GameState`) y crea una matriz de objetos `Tile`. Proporciona `draw(screen)` que itera sobre todas las celdas, pinta rectángulos con colores según el tipo y añade un borde sutil. Esto permite visualizar el camino, las zonas construibles (pasto) y los obstáculos (rocas).
- El layout está diseñado para que haya una entrada (izquierda), un camino que serpentea (por ahora solo horizontal en la fila 7 para simplificar) y el bastión al final (derecha). Las celdas adyacentes al camino son de tipo pasto, es decir, construibles. Los bordes son rocas no transitables.

### Camino (`Path`)

- Contiene una lista de `pygame.Vector2` que representan los puntos centrales de las celdas del camino. Esta lista se extrae automáticamente del layout: se recorren todas las celdas y si son `TILE_PATH` o `TILE_BASTION`, se añade su centro. En este ejemplo, el orden de inserción es por filas y columnas, lo que para un camino lineal de izquierda a derecha en la misma fila es correcto. En el futuro, cuando el camino tenga giros, se deberá definir manualmente el orden de los waypoints o utilizar un algoritmo de ordenación.

### Bastión (`Bastion`)

- Se instancia a partir de la posición obtenida de `Grid.get_bastion_position()`, que busca la celda con `TILE_BASTION`.
- Tiene atributos de vida (`health`, `max_health`) y métodos `take_damage()` e `is_destroyed()`. Se dibuja como un círculo gris con un borde y una barra de vida sobre él. Esto servirá como representación visual del objetivo a defender.

### Integración en `GameState`

- **Constructor**: define la matriz `layout`, crea `Grid` y `Path`, coloca el bastión. Esta es la “carga de nivel” básica. En el futuro, esta información vendrá de archivos JSON.
- **render()**: limpia la pantalla, dibuja la cuadrícula, el bastión y un texto informativo.
- **handle\_events()**: mantiene la tecla ESC para pausar (usa el mismo `PauseState` anterior).

### Principios de diseño

- **Modularidad**: el mapa, el camino y el bastión están en módulos separados. `GameState` solo los compone.
- **Escalabilidad**: añadir nuevos niveles solo requiere cambiar la matriz `layout` y posiblemente los waypoints.
- **Bajo acoplamiento**: `Grid` no conoce a los enemigos ni a las defensas; solo gestiona los tipos de celda. `Path` solo almacena puntos. El bastión es una entidad independiente.
- **Reutilización**: las clases `Tile` y `Grid` se usarán más adelante para validar posiciones de construcción y movimientos de enemigos.

---

## ## 4. Instrucciones de Ejecución e Integración

1. **Requisitos**: Python 3 y Pygame instalados.
2. **Estructura de archivos**: asegurarse de que todos los archivos estén en las rutas indicadas. Los `__init__.py` permiten importar los paquetes.
3. **Ejecución**: desde la carpeta `el-ultimo-bastion/` ejecutar `python main.py`.
4. **Controles**:
  - **Menú principal**: ENTER para jugar, ESC para salir.
  - **Juego**: se muestra el escenario con la cuadrícula, el camino marrón, el pasto verde y el bastión a la derecha. ESC para pausar.
  - **Pausa**: C para continuar, M para menú.
5. **Resultado visual esperado**: Una ventana de 1024×768 con una cuadrícula de 16×12 celdas. Las celdas exteriores son rocas (gris oscuro). En la séptima fila (índice 7) hay un camino horizontal marrón que va desde la columna 1 hasta la 12; la columna 13 es el bastión (círculo gris con barra de vida). El resto de celdas interiores son pasto verde (zonas construibles). En la esquina superior izquierda un texto "Escenario base - ESC para pausa".

---

### ## 5. Captura Conceptual del Resultado Esperado (Descripción)

...

```

+-----+
| R R R R R R R R R R R R R R R R |
| R G G G G P G G G G G G G G G R |
| R G G G G P G G G G G G G G G R |
| R G G G G P G G G G G G G G G R |
| R G G G G P G G G G G G G G G R |
| R G G G G P G G G G G G G G G R |
| R G G G G P G G G G G G G G G R |
| R P P P P P P P P P P P P B G R |
| R G G G G G G G G G G G G G G R |
| R G G G G G G G G G G G G G G R |
| R G G G G G G G G G G G G G G R |
| R R R R R R R R R R R R R R R R |
+-----+

```

R = Roca (gris), G = Pasto (verde), P = Camino (marrón), B = Bastión (círculo gris)  
 Camino: inicia en (1,7) y termina en (13,7) donde está el Bastión.

...

Este escenario constituye la base sobre la que se añadirán enemigos, torres y oleadas en las siguientes fases.